



Computer architecture

In [computer science](#) and [computer engineering](#), a **computer architecture** is the structure of a [computer system](#) made from component parts.^[1] It can sometimes be a high-level description that ignores details of the implementation.^[2] At a more detailed level, the description may include the [instruction set architecture](#) design, [microarchitecture](#) design, [logic design](#), and [implementation](#).^[3]

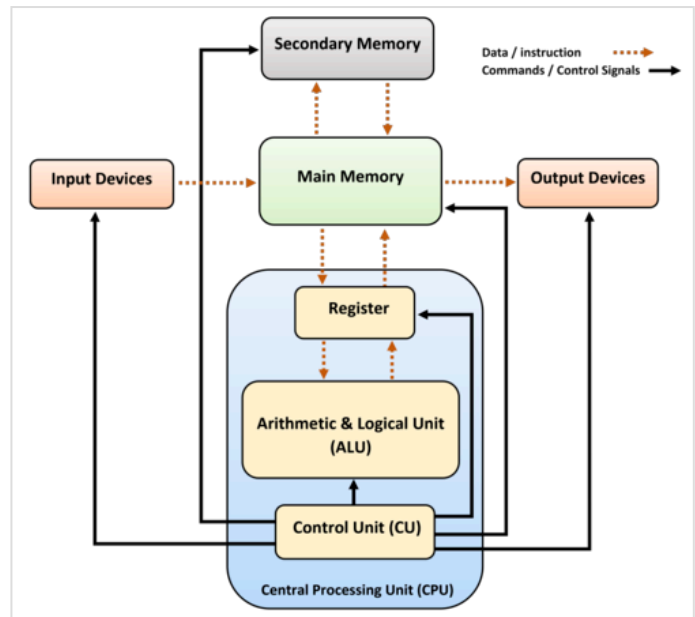
History

The first documented computer architecture was in the correspondence between [Charles Babbage](#) and [Ada Lovelace](#), describing the [analytical engine](#). While building the computer [Z1](#) in 1936, [Konrad Zuse](#) described in two patent applications for his future projects that machine instructions could be stored in the same storage used for data, i.e., the [stored-program](#) concept.^{[4][5]} Two other early and important examples are:

- [John von Neumann's 1945 paper, First Draft of a Report on the EDVAC](#), which described an organization of logical elements;^[6] and
- [Alan Turing's more detailed *Proposed Electronic Calculator* for the Automatic Computing Engine](#), also 1945 and which cited [John von Neumann's paper](#).^[7]

The term "architecture" in computer literature can be traced to the work of [Lyle R. Johnson](#) and [Frederick P. Brooks, Jr.](#), members of the Machine Organization department in IBM's main research center in 1959. Johnson had the opportunity to write a proprietary research communication about the [Stretch](#), an IBM-developed [supercomputer](#) for [Los Alamos National Laboratory](#) (at the time known as Los Alamos Scientific Laboratory). To describe the level of detail for discussing the luxuriously embellished computer, he noted that his description of formats, instruction types, hardware parameters, and speed enhancements were at the level of "system architecture", a term that seemed more useful than "machine organization".^[8]

Subsequently, Brooks, a Stretch designer, opened Chapter 2 of a book called *Planning a Computer System: Project Stretch* by stating, "Computer architecture, like other architecture, is the art of determining the needs of the user of a structure and then designing to meet those needs as effectively as possible within economic and technological constraints."^[9]



Block diagram of a basic computer with uniprocessor CPU. Black lines indicate the flow of control signals, whereas red lines indicate the flow of processor instructions and data. Arrows indicate the direction of flow.

Brooks went on to help develop the IBM System/360 line of computers, in which "architecture" became a noun defining "what the user needs to know".^[10] The System/360 line was succeeded by several compatible lines of computers, including the current IBM Z line. Later, computer users came to use the term in many less explicit ways.^[11]

The earliest computer architectures were designed on paper and then directly built into the final hardware form.^[12] Later, computer architecture prototypes were physically built in the form of a transistor-transistor logic (TTL) computer—such as the prototypes of the 6800 and the PA-RISC—tested, and tweaked, before committing to the final hardware form. As of the 1990s, new computer architectures are typically "built", tested, and tweaked—inside some other computer architecture in a computer architecture simulator; or inside a FPGA as a soft microprocessor; or both—before committing to the final hardware form.^[13]

Subcategories

The discipline of computer architecture has three main subcategories:^[14]

- **Instruction set architecture** (ISA): defines the machine code that a processor reads and acts upon as well as the word size, memory address modes, processor registers, and data type.
- **Microarchitecture**: also known as "computer organization", this describes how a particular processor will implement the ISA.^[15] The size of a computer's CPU cache for instance, is an issue that generally has nothing to do with the ISA.
- **Systems design**: includes all of the other hardware components within a computing system, such as data processing other than the CPU (e.g., direct memory access), virtualization, and multiprocessing.

There are other technologies in computer architecture. The following technologies are used in bigger companies like Intel, and were estimated in 2002^[14] to count for 1% of all of computer architecture:

- **Macroarchitecture**: architectural layers more abstract than microarchitecture
- **Assembly instruction set architecture**: A smart assembler may convert an abstract assembly language common to a group of machines into slightly different machine language for different implementations.
- **Programmer-visible macroarchitecture**: higher-level language tools such as compilers may define a consistent interface or contract to programmers using them, abstracting differences between underlying ISAs and microarchitectures. For example, the C, C++, or Java standards define different programmer-visible macroarchitectures.
- **Microcode**: microcode is software that translates instructions to run on a chip. It acts like a wrapper around the hardware, presenting a preferred version of the hardware's instruction set interface. This instruction translation facility gives chip designers flexible options: E.g. 1. A new improved version of the chip can use microcode to present the exact same instruction set as the old chip version, so all software targeting that instruction set will run on the new chip without needing changes. E.g. 2. Microcode can present a variety of instruction sets for the same underlying chip, allowing it to run a wider variety of software.
- **Pin architecture**: The hardware functions that a microprocessor should provide to a hardware platform, e.g., the x86 pins A20M, FERR/IGNNE or FLUSH. Also, messages that the processor should emit so that external caches can be invalidated (emptied). Pin architecture functions are more flexible than ISA functions because external hardware can adapt to new encodings, or change from a pin to a message. The term "architecture" fits,

because the functions must be provided for compatible systems, even if the detailed method changes.

Roles

Definition

Computer architecture is concerned with balancing the performance, efficiency, cost, and reliability of a computer system. The case of instruction set architecture can be used to illustrate the balance of these competing factors. More complex instruction sets enable programmers to write more space efficient programs, since a single instruction can encode some higher-level abstraction (such as the x86 Loop instruction).^[16] However, longer and more complex instructions take longer for the processor to decode and can be more costly to implement effectively. The increased complexity from a large instruction set also creates more room for unreliability when instructions interact in unexpected ways.

The implementation involves integrated circuit design, packaging, power, and cooling. Optimization of the design requires familiarity with topics from compilers and operating systems to logic design and packaging.^[17]

Instruction set architecture

An instruction set architecture (ISA) is the interface between the computer's software and hardware and also can be viewed as the programmer's view of the machine. Computers do not understand high-level programming languages such as Java, C++, or most programming languages used. A processor only understands instructions encoded in some numerical fashion, usually as binary numbers. Software tools, such as compilers, translate those high level languages into instructions that the processor can understand.^{[18][19]}

Besides instructions, the ISA defines items in the computer that are available to a program—e.g., data types, registers, addressing modes, and memory. Instructions locate these available items with register indexes (or names) and memory addressing modes.^{[20][21]}

The ISA of a computer is usually described in a small instruction manual, which describes how the instructions are encoded. Also, it may define short (vaguely) mnemonic names for the instructions. The names can be recognized by a software development tool called an assembler. An assembler is a computer program that translates a human-readable form of the ISA into a computer-readable form. Disassemblers are also widely available, usually in debuggers and software programs to isolate and correct malfunctions in binary computer programs.^[22]

ISAs vary in quality and completeness. A good ISA compromises between programmer convenience (how easy the code is to understand), size of the code (how much code is required to do a specific action), cost of the computer to interpret the instructions (more complexity means more hardware needed to decode and execute the instructions), and speed of the computer (with more complex decoding hardware comes longer decode time). Memory organization defines how instructions interact with the memory, and how memory interacts with itself.

During design emulation, emulators can run programs written in a proposed instruction set. Modern emulators can measure size, cost, and speed to determine whether a particular ISA is meeting its goals.

Computer organization

Computer organization helps optimize performance-based products. For example, software engineers need to know the processing power of processors. They may need to optimize software in order to gain the most performance for the lowest price. This can require quite a detailed analysis of the computer's organization. For example, in an SD card, the designers might need to arrange the card so that the most data can be processed in the fastest possible way.

Computer organization also helps plan the selection of a processor for a particular project. Multimedia projects may need very rapid data access, while virtual machines may need fast interrupts. Sometimes certain tasks need additional components as well. For example, a computer capable of running a virtual machine needs virtual memory hardware so that the memory of different virtual computers can be kept separated. Computer organization and features also affect power consumption and processor cost.

Implementation

Once an instruction set and microarchitecture have been designed, a practical machine must be developed. This design process is called the *implementation*. Implementation is usually not considered architectural design, but rather hardware design engineering. Implementation can be further broken down into several steps:

- **Logic implementation** designs the circuits required at a logic-gate level.
- **Circuit implementation** does transistor-level designs of basic elements (e.g., gates, multiplexers, latches) as well as of some larger blocks (ALUs, caches etc.) that may be implemented at the logic-gate level, or even at the physical level if the design calls for it.
- **Physical implementation** draws physical circuits. The different circuit components are placed in a chip floor plan or on a board and the wires connecting them are created.
- **Design validation** tests the computer as a whole to see if it works in all situations and all timings. Once the design validation process starts, the design at the logic level are tested using logic emulators. However, this is usually too slow to run a realistic test. So, after making corrections based on the first test, prototypes are constructed using Field-Programmable Gate-Arrays (FPGAs). Most hobby projects stop at this stage. The final step is to test prototype integrated circuits, which may require several redesigns.

For CPUs, the entire implementation process is organized differently and is often referred to as CPU design.

Design goals

The exact form of a computer system depends on the constraints and goals. Computer architectures usually trade off standards, power versus performance, cost, memory capacity, latency (latency is the amount of time that it takes for information from one node to travel to the source) and throughput. Sometimes other considerations, such as features, size, weight, reliability, and expandability are also factors.

The most common scheme does an in-depth power analysis and figures out how to keep power consumption low while maintaining adequate performance.

Performance

Modern computer performance is often described in instructions per cycle (IPC), which measures the efficiency of the architecture at any clock frequency; a higher IPC means the computer can do more work per unit of time. Older computers had IPC counts as low as 0.1 while modern processors easily reach nearly 1. Superscalar processors may reach three to five IPC by executing several instructions per clock cycle.

Counting machine-language instructions would be misleading because they can do varying amounts of work in different ISAs. The "instruction" in the standard measurements is not a count of the ISA's machine-language instructions, but a unit of measurement, usually based on the speed of the VAX computer architecture.

Many people used to measure a computer's speed by the clock rate (usually in MHz or GHz). This refers to the cycles per second of the main clock of the CPU. However, this metric is somewhat misleading, as a machine with a higher clock rate may not necessarily have greater performance. As a result, manufacturers have moved away from clock speed as a measure of performance.

Other factors influence speed, such as the mix of functional units, bus speeds, available memory, and the type and order of instructions in the programs.

There are two main types of speed: latency and throughput. Latency is the time between the start of a process and its completion. Throughput is the amount of work done per unit time. Interrupt latency is the guaranteed maximum response time of the system to an electronic event (like when the disk drive finishes moving some data).

Performance is affected by a very wide range of design choices — for example, pipelining a processor usually makes latency worse, but makes throughput better. Computers that control machinery usually need low interrupt latencies. These computers operate in a real-time environment and fail if an operation is not completed in a specified amount of time. For example, computer-controlled anti-lock brakes must begin braking within a predictable and limited time period after the brake pedal is sensed or else failure of the brake will occur.

Benchmarking takes all these factors into account by measuring the time a computer takes to run through a series of test programs. Although benchmarking shows strengths, it should not be how you choose a computer. Often the measured machines split on different measures. For example, one system might handle scientific applications quickly, while another might render video games more smoothly. Furthermore, designers may target and add special features to their products, through hardware or software, that permit a specific benchmark to execute quickly but do not offer similar advantages to general tasks.

Power efficiency

Power efficiency is another important measurement in modern computers. Higher power efficiency can often be traded for lower speed or higher cost. Historically, the typical measurement when referring to power consumption in computer architecture was MIPS/W (millions of instructions per second per watt), though modern designs use more sophisticated measurements such as performance-per-watt benchmarks and energy-per-instruction.

Modern integrated circuits consume more total power despite improvements in per-transistor energy efficiency achieved through process scaling.^[23] This increase stems from the rapid growth in transistor count per die, which demands increasingly complex power delivery networks.^[24] Additionally, power density (watts per unit area) rises as feature sizes (e.g., transistor size) shrink, making thermal management more challenging.^[25] Elevated operating temperatures further complicate these thermal challenges by increasing interconnect resistance, creating a positive feedback loop between power consumption and heat generation.

As transistor scaling has slowed and power constraints have become limiting factors, power efficiency is starting to become as important, if not more important than fitting more and more transistors into a single chip. Recent processor designs have shown this emphasis as they put more focus on power efficiency rather than cramming as many transistors into a single chip as possible.^[26] In the world of embedded computers, power efficiency has long been an important goal next to throughput and latency.

Shifts in market demand

Increases in clock frequency have grown more slowly over the past few years, compared to power reduction improvements. This has been driven by the end of Moore's Law and demand for longer battery life and reductions in size for mobile technology. This change in focus from higher clock rates to power consumption and miniaturization can be shown by the significant reductions in power consumption, as much as 50%, that were reported by Intel in their release of the Haswell microarchitecture; where they dropped their power consumption benchmark from 30–40 watts down to 10–20 watts.^[27] Comparing this to the processing speed increase of 3 GHz to 4 GHz (2002 to 2006), it can be seen that the focus in research and development is shifting away from clock frequency and moving towards consuming less power and taking up less space.^[28]

See also



Electronics portal

- Bit-serial architecture
- Comparison of CPU architectures
- Computer hardware
- CPU design
- Dataflow architecture
- Floating point
- Flynn's taxonomy
- Harvard architecture (Modified)

- Influence of the IBM PC on the personal computer market
- Orthogonal instruction set
- Reconfigurable computing
- Software architecture
- Transport triggered architecture
- Von Neumann architecture

References

1. Dragoni, Nicole (n.d.). "Introduction to peer to peer computing" (<http://www2.imm.dtu.dk/courses/02220/2017/L6/P2P.pdf>) (PDF). *DTU Compute – Department of Applied Mathematics and Computer Science*. Lyngby, Denmark.
2. Clements, Alan. *Principles of Computer Hardware* (Fourth ed.). p. 1. "Architecture describes the internal organization of a computer in an abstract way; that is, it defines the capabilities of the computer and its programming model. You can have two computers that have been constructed in different ways with different technologies but with the same architecture."
3. Hennessy, John; Patterson, David. *Computer Architecture: A Quantitative Approach* (Fifth ed.). p. 11. "This task has many aspects, including instruction set design, functional organization, logic design, and implementation."
4. Williams, F. C.; Kilburn, T. (25 September 1948), "Electronic Digital Computers", *Nature*, **162** (4117): 487, Bibcode:1948Natur.162..487W (<https://ui.adsabs.harvard.edu/abs/1948Natur.162..487W>), doi:10.1038/162487a0 (<https://doi.org/10.1038%2F162487a0>), S2CID 4110351 (<https://api.semanticscholar.org/CorpusID:4110351>)
5. Susanne Faber, "Konrad Zuses Bemuehungen um die Patentanmeldung der Z3", 2000
6. Neumann, John (1945). *First Draft of a Report on the EDVAC*. p. 9.
7. Reproduced in B. J. Copeland (Ed.), "Alan Turing's Automatic Computing Engine", Oxford University Press, 2005, pp. 369–454.
8. Johnson, Lyle (1960). "A Description of Stretch" (<https://archive.computerhistory.org/resources/text/IBM/Stretch/pdfs/05-10/102634114.pdf>) (PDF). p. 1. Retrieved 7 October 2017.
9. Buchholz, Werner (1962). *Planning a Computer System*. p. 5.
10. "System 360, From Computers to Computer Systems" (<https://web.archive.org/web/20120403020049/http://www-03.ibm.com/ibm/history/ibm100/us/en/icons/system360/>). *IBM100*. 7 March 2012. Archived from the original (<http://www-03.ibm.com/ibm/history/ibm100/us/en/icons/system360/>) on April 3, 2012. Retrieved 11 May 2017.
11. Hellige, Hans Dieter (2004). "Die Genese von Wissenschaftskonzeptionen der Computerarchitektur: Vom "system of organs" zum Schichtmodell des Designraums". *Geschichten der Informatik: Visionen, Paradigmen, Leit motive*. pp. 411–472.
12. ACE underwent seven paper designs in one year, before a prototype was initiated in 1948. [B. J. Copeland (Ed.), "Alan Turing's Automatic Computing Engine", OUP, 2005, p. 57]
13. Schmalz, M.S. "Organization of Computer Systems" (<https://www.cise.ufl.edu/~mssz/CompOrg/CDAintro.html>). *UF CISE*. Retrieved 11 May 2017.
14. John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach* (Third ed.). Morgan Kaufmann Publishers.
15. Laplante, Phillip A. (2001). *Dictionary of Computer Science, Engineering, and Technology*. CRC Press. pp. 94–95. ISBN 0-8493-2691-5.
16. Null, Linda (2019). *The Essentials of Computer Organization and Architecture* (5th ed.). Burlington, MA: Jones & Bartlett Learning. p. 280. ISBN 9781284123036.
17. Martin, Milo. "What is computer architecture?" (https://www.cis.upenn.edu/~milom/cis501-Fall11/lectures/00_intro.pdf) (PDF). *UPENN*. Retrieved 11 May 2017.

18. "Glossary" (<https://codasip.com/glossary/isa>). *Codasip*. Retrieved 30 May 2025.
19. "What is Instruction Set Architecture (ISA)?" (<https://www.arm.com/glossary/isa>). *The Architecture for the Digital World*. Retrieved 30 May 2025.
20. "Organization of Computer Systems: ISA, Machine Language, Number Systems" (<https://www.cise.ufl.edu/~mssz/CompOrg/CDA-lang.html>). *www.cise.ufl.edu*. Retrieved 30 May 2025.
21. "Instruction Set Architecture – Computer Architecture" (<https://www.cs.umd.edu/~meesh/411/CA-online/chapter/instruction-set-architecture/index.html>). *www.cs.umd.edu*. Retrieved 30 May 2025.
22. Hennessy, John L.; Patterson, David A. (23 November 2017). *Computer Architecture: A Quantitative Approach* (https://google.com/books/edition/Computer_Architecture/cM8mDwAAQBAJ). Morgan Kaufmann Publishers. ISBN 978-0-12-811906-8. Retrieved 30 May 2025.
23. "International Roadmap for Devices and Systems: 2022 Edition Executive Summary" (https://irds.ieee.org/images/files/pdf/2022/2022IRDS_ES.pdf) (PDF). *IEEE*. Retrieved 7 January 2026.
24. Radhakrishnan, Kaladhar; Swaminathan, Madhavan; Bhattacharyya, Bidyut K. "Power Delivery for High-Performance Microprocessors—Challenges, Solutions, and Future Trends" (<https://ieeexplore.ieee.org/document/9377004/>). *IEEE Transactions on Components, Packaging and Manufacturing Technology*. **11** (4): 655–671. doi:10.1109/TCPMT.2021.3065690 (<https://doi.org/10.1109%2FTCPMT.2021.3065690>). ISSN 2156-3985 (<https://search.worldcat.org/issn/2156-3985>).
25. Banerjee, Kaustav; Mehrotra, Amit (2001). "Global (Interconnect) Warming" (https://web.ecs.ucsb.edu/Faculty/Banerjee/pubs/IEEE_Ckts_Dev_Mag_2001.pdf) (PDF). Retrieved 7 January 2026.
26. "Exynos 9 Series (8895)" (http://www.samsung.com/semiconductor/minisite/Exynos/w/solution/mod_ap/8895/?CID=AFL-hq-mul-0813-11000170). *Samsung*. Retrieved 8 May 2017.
27. "Measuring Processor Power TDP vs ACP" (<http://www.intel.com/content/dam/doc/white-paper/resources-xeon-measuring-processor-power-paper.pdf>) (PDF). *Intel*. April 2011. Retrieved 5 May 2017.
28. "History of Processor Performance" (<https://www.cs.columbia.edu/~sedwards/classes/2012/3827-spring/advanced-arch-2011.pdf>) (PDF). *cs.columbia.edu*. 24 April 2012. Retrieved 5 May 2017.

Sources

- John L. Hennessy and David Patterson (2006). *Computer Architecture: A Quantitative Approach* (<http://www.elsevierdirect.com/product.jsp?isbn=9780123704900>) (Fourth ed.). Morgan Kaufmann. ISBN 978-0-12-370490-0.
- Barton, Robert S., "Functional Design of Computers", *Communications of the ACM* 4(9): 405 (1961).
- Barton, Robert S., "A New Approach to the Functional Design of a Digital Computer", *Proceedings of the Western Joint Computer Conference*, May 1961, pp. 393–396. About the design of the Burroughs B5000 computer.
- Bell, C. Gordon; and Newell, Allen (1971). "Computer Structures: Readings and Examples" (http://research.microsoft.com/en-us/um/people/gbell/Computer_Structures_Readings_and_Examples/contents.html), McGraw-Hill.
- Blaauw, G.A., and Brooks, F.P., Jr., "The Structure of System/360, Part I—Outline of the Logical Structure" (<http://domino.research.ibm.com/tchjr/journalindex.nsf/d9f0a910ab8b637485256bc80066a393/95dc427e3fd3024a85256bfa006859f7?OpenDocument>), *IBM Systems Journal*, vol. 3, no. 2, pp. 119–135, 1964.

- Tanenbaum, Andrew S. (1979). *Structured Computer Organization*. Englewood Cliffs, New Jersey: Prentice-Hall. ISBN 0-13-148521-0.

External links

- Carnegie Mellon Computer Architecture Lectures (<https://www.youtube.com/user/cmu18447>)
- ISCA: Proceedings of the International Symposium on Computer Architecture (http://portal.acm.org/toc.cfm?id=SERIES416&type=series&coll=GUIDE&dl=GUIDE&CFID=41492512&CF_TOKEN=82922478)
- Micro: IEEE/ACM International Symposium on Microarchitecture (<http://www.microarch.org/>)
- HPCA: International Symposium on High Performance Computer Architecture (<https://web.archive.org/web/20050528085407/http://www.hpcacnf.org/>)
- ASPLOS: International Conference on Architectural Support for Programming Languages and Operating Systems (<http://portal.acm.org/toc.cfm?id=SERIES311&type=series&coll=GUIDE&dl=GUIDE&CFID=41492415&CFTOKEN=3676847>)
- ACM Transactions on Architecture and Code Optimization (<http://www.acm.org/taco/>)
- IEEE Transactions on Computers (<https://www.computer.org/csdl/journal/tc>)
- The von Neumann Architecture of Computer Systems (<https://web.archive.org/web/20171031185802/http://www-scf.usc.edu/~inf520/downloads/The%20von%20Neumann%20Architecture%20of%20Computer%20Systems.pdf>) at the [Wayback Machine](#) (archived 2017-10-31)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Computer_architecture&oldid=1332476510"